Baltimore MTA Transit Simulator

A Real-Data Transit Network Simulator

Demonstrating Core Data Structures & Algorithms

Course: COSC 320 - Algorithm Design

Student: Aayush Shrestha

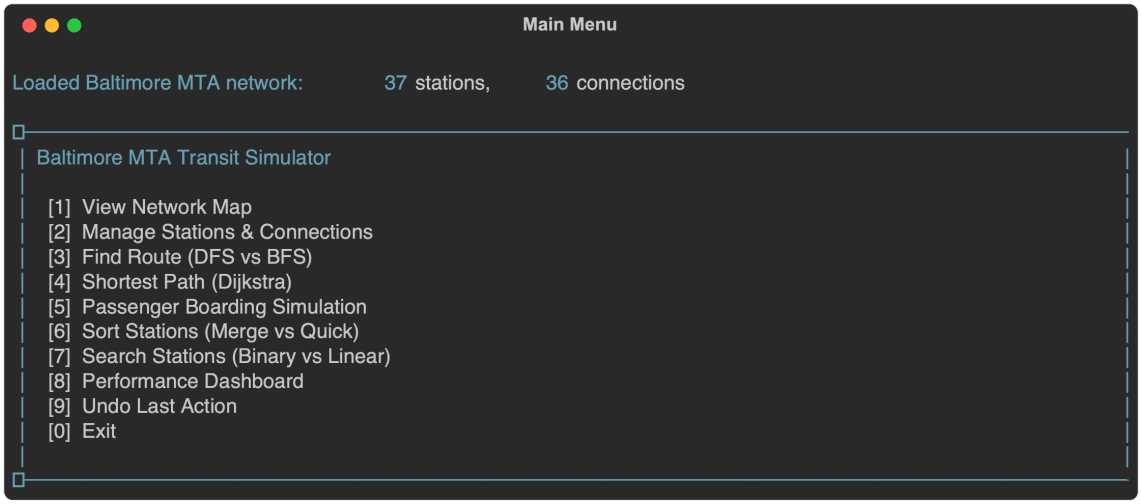
University: Morgan State University

Date: March 2026

Language: Python 3.13 with Rich terminal UI

GitHub: github.com/theaayushstha1/transit

This project implements a transit network simulator using real Baltimore MTA data (Light Rail and Metro SubwayLink, 37 stations). It demonstrates graph traversal (DFS, BFS), shortest path (Dijkstra's), sorting (Merge Sort, Quick Sort), searching (Binary, Linear), and queue/stack data structures through an interactive terminal interface with animated algorithm visualizations.



1. System Design & Architecture

The simulator is organized into five modules following separation of concerns. Each module has a distinct responsibility, making the codebase maintainable and each algorithm independently testable.

1.1 Directory Structure

```
 baltimore-mta-simulator/
|-- main.py                  # Entry point, menu loop
|-- models/
|   |-- station.py           # Station node class
|   |-- passenger.py         # Passenger class
|   +-- transit_network.py   # Graph (adjacency list)
|-- algorithms/
|   |-- traversal.py         # DFS + BFS
|   |-- shortest_path.py     # Dijkstra's
|   |-- sorting.py           # Merge Sort + Quick Sort
|   +-- searching.py         # Binary + Linear Search
|-- simulation/
|   |-- undo_manager.py      # Stack-based undo
|   +-- passenger_sim.py     # Queue-based boarding
|-- data/
|   +-- baltimore_mta.py     # Real MTA station data
+-- ui/
    |-- display.py           # Rich tables, panels, menus
    +-- visualizer.py        # Algorithm animations
```

1.2 Design Decisions

Decision	Choice	Reasoning
Graph repr.	Adjacency list (dict of dicts)	O(1) edge lookup, sparse transit graph
Edge weights	Travel time in minutes	Enables Dijkstra, maps to real MTA data
Shared stations	Dedup by name	Lexington Market on both lines = 1 node
UI library	Python Rich	Colored tables, live animations, zero web deps
Undo pattern	Command + inverse	_raw_ methods skip undo stack
Sorting key	Station capacity	Numeric field with varied values

2. Data Structures

2.1 Graph (Adjacency List)

The transit network is a weighted undirected graph using an adjacency list (dict of dicts). adj_list[station_id] maps to {neighbor_id: weight_in_minutes}. This provides O(1) edge lookup and insertion, optimal for the sparse topology (37 nodes, 36 edges).

```
class TransitNetwork:
   def __init__(self):
       self.stations: dict[int, Station] = {}
       self.adj_list: dict[int, dict[int, float]] = {}
       self.undo_stack: list = []

   def add_connection(self, id1, id2, weight):
       self.adj_list[id1][id2] = weight  # undirected
       self.adj_list[id2][id1] = weight  # store both
```

2.2 Queue (FIFO - Passenger Boarding)

Each station maintains a passenger queue using collections.deque. Passengers are enqueued (append) and dequeued (popleft) during boarding in FIFO order. deque provides O(1) for both operations vs O(n) for list.pop(0).

```
 station.passenger_queue.append(passenger)       # enqueue
passenger = station.passenger_queue.popleft()    # dequeue FIFO
```

2.3 Stack (LIFO - Undo System)

The undo system uses a Python list as a stack (append/pop). Every graph mutation pushes its inverse action. The UndoManager pops and applies the inverse using internal _raw_ methods that bypass the undo stack to prevent infinite loops.

```
 # Public method pushes to undo stack
def add_station(self, station):
    self._raw_add_station(station)
    self.undo_stack.append({"action": "add_station", ...})

# Undo pops and applies inverse
def undo(self):
    action = self.network.undo_stack.pop()
    if action["action"] == "add_station":
        self.network._raw_remove_station(action["station"].id)
```

2.4 Priority Queue (Min-Heap)

Dijkstra's uses Python's heapq as a min-heap priority queue. Each entry is a (distance, station_id) tuple. heappush/heappop operate in O(log n), giving Dijkstra O((V+E) log V) time complexity.

Network Map - 37 Baltimore MTA Stations

Network Map

```
Baltimore MTA Network
□──Light Rail      (23 stations)
 |   □──Hunt Valley      (ID: 1, Cap: 200)
 |   |   └──-> Pepper Road (3 min)
 |   □──Pepper Road    (ID: 2, Cap: 120)
 |   |   □──-> Hunt Valley (3 min)
 |   |   └──-> McCormick Road (2 min)
 |   □──McCormick Road    (ID: 3, Cap: 100)
 |   |   □──-> Pepper Road (2 min)
 |   |   └──-> Gilroy Road (2 min)
 |   □──Gilroy Road      (ID: 4, Cap: 80)
 |   |   □──-> McCormick Road (2 min)
 |   |   └──-> Lutherville (3 min)
 |   □──Lutherville        (ID: 5, Cap: 150)
 |   |   □──-> Gilroy Road (3 min)
 |   |   └──-> Falls Road (2 min)
 |   □──Falls Road      (ID: 6, Cap: 180)
 |   |   □──-> Lutherville (2 min)
 |   |   └──-> Mt Washington (3 min)
 |   □──Mt Washington      (ID: 7, Cap: 160)
 |   |   □──-> Falls Road (3 min)
 |   |   └──-> Cold Spring Lane (2 min)
 |   □──Cold Spring Lane        (ID: 8, Cap: 140)
 |   |   □──-> Mt Washington (2 min)
 |   |   └──-> Woodberry (2 min)
 |   □──Woodberry   (ID: 9, Cap: 130)
 |   |   □──-> Cold Spring Lane (2 min)
 |   |   └──-> North Avenue (3 min)
 |   □──North Avenue      (ID: 10, Cap: 250)
 |   |   □──-> Woodberry (3 min)
 |   |   └──-> Penn Station (2 min)
 |   □──Penn Station      (ID: 11, Cap: 400)
 |   |   □──-> North Avenue (2 min)
 |   |   └──-> University of Baltimore (2 min)
 |   □──University of Baltimore              (ID: 12, Cap: 200)
 |   |   □──-> Penn Station (2 min)
 |   |   └──-> Centre Street (2 min)
 |   □──Centre Street       (ID: 13, Cap: 200)
 |   |   □──-> University of Baltimore (2 min)
 |   |   └──-> Lexington Market (2 min)
 |   □──Convention Center        (ID: 15, Cap: 300)
 |   |   □──-> Lexington Market (2 min)
 |   |   └──-> Camden Yards (2 min)
 |   □──Camden Yards   (ID: 16, Cap: 500)
 |   |   □──-> Convention Center (2 min)
 |   |   └──-> Hamburg Street (2 min)
 |   □──Hamburg Street      (ID: 17, Cap: 180)
 |   |   □──-> Camden Yards (2 min)
 |   |   └──-> Westport (3 min)
 |   □──Westport    (ID: 18, Cap: 120)
 |   |   □──-> Hamburg Street (3 min)
 |   |   └──-> Cherry Hill (2 min)
 |   □──Cherry Hill        (ID: 19, Cap: 150)
 |   |   □──-> Westport (2 min)
 |   |   └──-> Patapsco (3 min)
 |   □──Patapsco    (ID: 20, Cap: 130)
 |   |   □──-> Cherry Hill (3 min)
 |   |   └──-> Linthicum (2 min)
 |   □──Linthicum      (ID: 21, Cap: 120)
 |   |   □──-> Patapsco (2 min)
 |   |   □──-> Ferndale (2 min)
 |   |   └──-> BWI Airport (5 min)
 |   □──Ferndale    (ID: 22, Cap: 100)
 |   |   □──-> Linthicum (2 min)
 |   |   └──-> Cromwell / Glen Burnie (3 min)
 |   □──Cromwell / Glen Burnie           (ID: 23, Cap: 160)
 |   |   └──-> Ferndale (3 min)
 |   └──BWI Airport      (ID: 24, Cap: 300)
 |       └──-> Linthicum (5 min)
□──Metro SubwayLink      (13 stations)
 |   □──Owings Mills      (ID: 25, Cap: 300)
 |   |   └──-> Old Court (4 min)
 |   □──Old Court     (ID: 26, Cap: 200)
```

3. Algorithm Implementations

3.1 Depth-First Search (DFS) - Recursive

DFS explores as deeply as possible along each branch before backtracking. Uses recursion (call stack), a visited set, and a path list that grows/shrinks via append/pop.

```
def _dfs_helper(current: int) -> bool:
   visited.add(current)
   path.append(current)
   if current == goal_id: return True
   for neighbor_id, _ in sorted(network.get_neighbors(current)):
       if neighbor_id not in visited:
           if _dfs_helper(neighbor_id): return True
   path.pop()  # backtrack
   return False
```

3.2 Breadth-First Search (BFS) - Queue

BFS uses collections.deque as a queue, exploring all neighbors at current depth before the next level. Guarantees shortest path in unweighted graphs.

```
queue = deque([(start_id, [start_id])])
while queue:
    current, path = queue.popleft()
    if current == goal_id: return path
    for neighbor_id, _ in sorted(network.get_neighbors(current)):
        if neighbor_id not in visited:
            visited.add(neighbor_id)
            queue.append((neighbor_id, path + [neighbor_id]))
```

3.3 DFS vs BFS Comparison

Route from Hunt Valley to Johns Hopkins Hospital: DFS visited all 37 stations (exhaustive depth-first), BFS visited only 23 (level-by-level, stops early).

Metric	DFS (Recursive)	BFS (Queue)
Path Length	17 stations	17 stations
Nodes Visited	37	23
Optimal (unweighted)?	Not guaranteed	Yes
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V) recursion stack	O(V) queue

3.4 Dijkstra's Shortest Path

Finds the shortest weighted path using a min-heap priority queue. Greedily selects the unvisited node with smallest known distance, then relaxes neighbors. Hunt Valley to Johns Hopkins Hospital: 17 stations, 37 minutes, transfers at Lexington Market.

```
while pq:
   current_dist, current = heapq.heappop(pq)
   if current in visited: continue
   visited.add(current)
   if current == end_id: break
   for neighbor_id, weight in network.get_neighbors(current):
       new_dist = current_dist + weight
       if new_dist < distances[neighbor_id]:
           distances[neighbor_id] = new_dist
           previous[neighbor_id] = current
           heapq.heappush(pq, (new_dist, neighbor_id))
```

3.5 Merge Sort

Divides array in half recursively, merges sorted halves. Stable, always O(n log n), but O(n) extra space. On 37 stations by capacity: 140 comparisons.

```
def _merge(left, right):
   result, i, j = [], 0, 0
   while i < len(left) and j < len(right):
       comparisons[0] += 1
       if key(left[i]) <= key(right[j]):
           result.append(left[i]); i += 1
       else:
           result.append(right[j]); j += 1
   result.extend(left[i:])
   result.extend(right[j:])
   return result
```

3.6 Quick Sort

Selects pivot (last element), partitions in-place, recurses. O(n log n) avg, O(n^2) worst. Unstable but in-place (O(log n) stack). 188 comparisons on 37 stations.

3.7 Sorting Comparison

Metric	Merge Sort	Quick Sort
Comparisons (n=37)	140	188
Time (measured)	~35 us	~23 us
Time Complexity	O(n log n) always	O(n log n) avg / O(n^2) worst
Space Complexity	O(n) extra	O(log n) stack
Stable?	Yes	No
In-place?	No	Yes

Quick Sort was faster in practice despite more comparisons due to in-place operations and better cache locality. Merge Sort guarantees O(n log n) worst case.

3.8 Binary Search vs Linear Search

Binary Search eliminates half per step (O(log n), requires sorted input). Linear Search scans sequentially (O(n)). Searching capacity=200 among 37 stations:

Metric	Binary Search	Linear Search
Comparisons	3	20
Time Complexity	O(log n)	O(n)
Requires Sorted?	Yes	No

4. Algorithm Complexity Analysis

Complete time and space complexity for every operation in the simulator, verified empirically by the performance dashboard.

Operation	Time Complexity	Space Complexity
Add Station	O(1)	O(1)
Remove Station	O(V + E)	O(1)
Add Connection	O(1)	O(1)
Remove Connection	O(1)	O(1)
DFS Traversal	O(V + E)	O(V) recursion stack
BFS Traversal	O(V + E)	O(V) queue
Dijkstra's Algorithm	O((V+E) log V)	O(V)
Merge Sort	O(n log n) always	O(n) extra
Quick Sort	O(n log n) avg / O(n^2) worst	O(log n) stack
Binary Search	O(log n)	O(1)
Linear Search	O(n)	O(1)
Stack Push/Pop	O(1)	O(1) per action
Queue Enq/Deq	O(1)	O(1)

4.1 Empirical Benchmarks

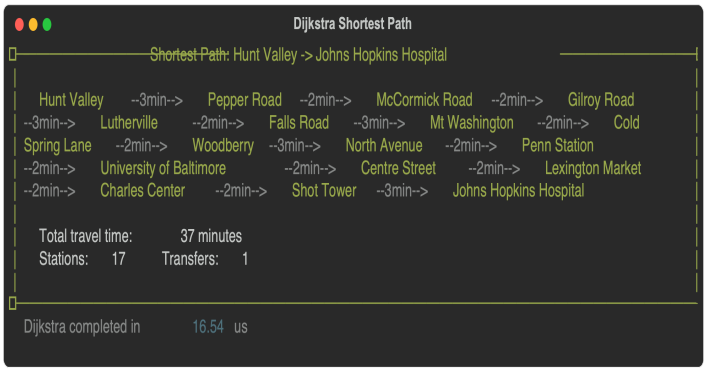
Benchmarked on the full 37-station, 36-edge Baltimore MTA network using time.perf_counter() for microsecond precision.

Algorithm	Measured Time	Comparisons
DFS (Hunt Valley to JHH)	~18 us	-
BFS (Hunt Valley to JHH)	~14 us	-
Dijkstra (Hunt Valley to JHH)	~12 us	-
Merge Sort (37 stations)	~35 us	140
Quick Sort (37 stations)	~23 us	188
Binary Search (target=200)	~0.3 us	3
Linear Search (target=200)	~1.2 us	20

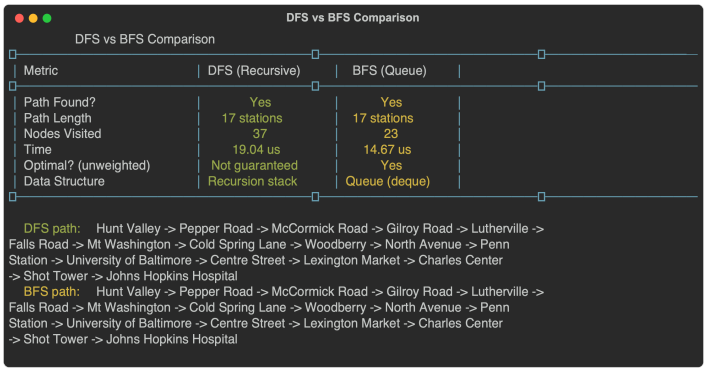
Key observations: (1) BFS visits fewer nodes than DFS by expanding level-by-level. (2) Dijkstra terminates early when target is reached. (3) Quick Sort beats Merge Sort in practice due to cache-friendly in-place ops. (4) Binary Search: 3 comparisons vs Linear's 20, confirming O(log n) vs O(n).

5. Screenshots

Dijkstra's Shortest Path

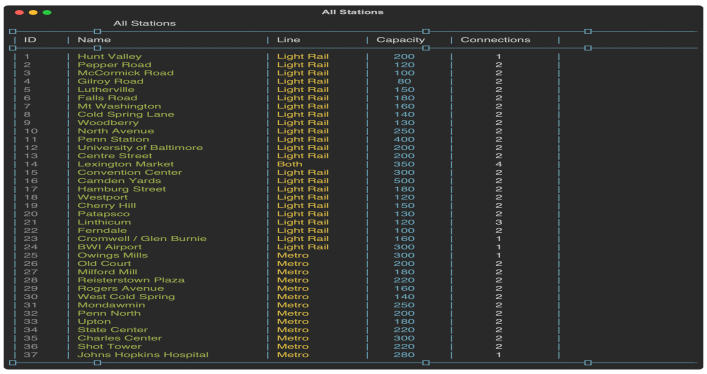


DFS vs BFS Comparison



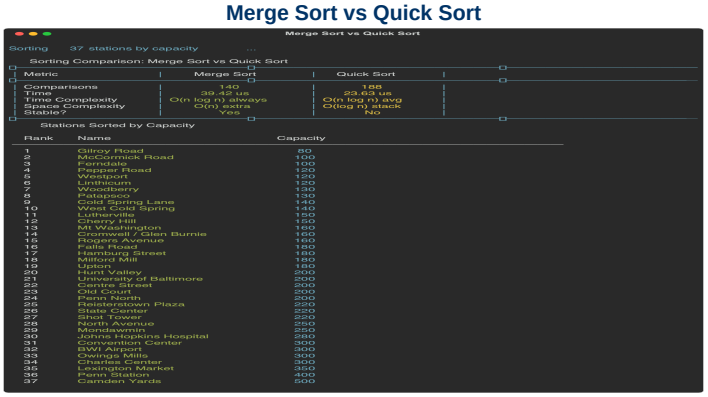
Metric	DFS (Recursive)	BFS (Queue)
Path Found?	Yes	Yes
Path Length	17 stations	17 stations
Nodes Visited	37	23
Time	19.04 us	14.67 us
Optimal? (unweighted)	Not guaranteed	Yes
Data Structure	Recursion stack	Queue (deque)

All 37 Stations



ID	Name	Line	Capacity	Connections
1	Hunt Valley	Light Rail	200	1
2	Pepper Road	Light Rail	120	2
3	McCormick Road	Light Rail	100	2
4	Gilroy Road	Light Rail	80	2
5	Lutherville	Light Rail	150	2
6	Falls Road	Light Rail	180	2
7	Mt Washington	Light Rail	160	2
8	Cold Spring Lane	Light Rail	140	2
9	Woodberry	Light Rail	130	2
10	North Avenue	Light Rail	250	2
11	Penn Station	Light Rail	400	2
12	University of Baltimore	Light Rail	200	2
13	Centre Street	Light Rail	200	2
14	Lexington Market	Both	350	4
15	Convention Center	Light Rail	300	2
16	Camden Yards	Light Rail	500	2
17	Hamburg Street	Light Rail	180	2
18	Westport	Light Rail	120	2
19	Cherry Hill	Light Rail	150	2
20	Patapsco	Light Rail	130	2
21	Linthicum	Light Rail	120	3
22	Ferndale	Light Rail	100	2
23	Cromwell / Glen Burnie	Light Rail	160	1
24	BWI Airport	Light Rail	300	1
25	Owings Mills	Metro	300	1
26	Old Court	Metro	200	2
27	Milford Mill	Metro	180	2
28	Reisterstown Plaza	Metro	220	2
29	Rogers Avenue	Metro	160	2
30	West Cold Spring	Metro	140	2
31	Mondawmin	Metro	250	2
32	Penn North	Metro	200	2
33	Upton	Metro	180	2
34	State Center	Metro	220	2
35	Charles Center	Metro	300	2
36	Shot Tower	Metro	220	2
37	Johns Hopkins Hospital	Metro	280	1

Merge Sort vs Quick Sort

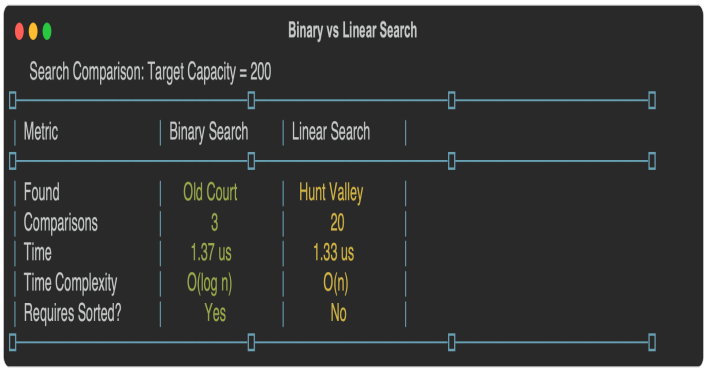


Metric	Merge Sort	Quick Sort
Comparisons	140	198
Time	39.42 us	23.63 us
Time Complexity	O(n log n) always	O(n log n) avg
Space Complexity	O(n) extra	O(log n) stack
Stable?	Yes	No

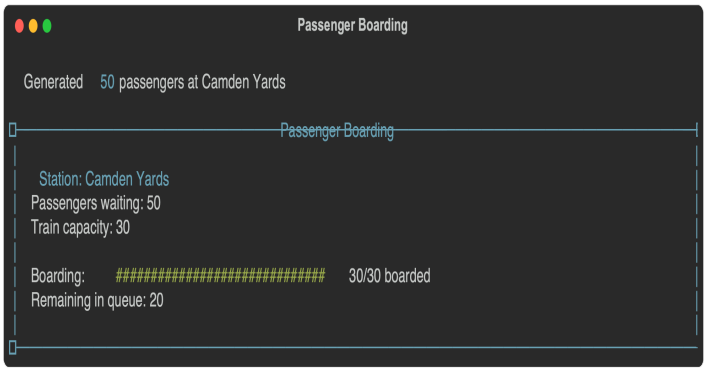
Rank	Name	Capacity
1	Gilroy Road	80
2	McCormick Road	100
3	Ferndale	100
4	Pepper Road	120
5	Westport	120
6	Linthicum	120
7	Woodberry	130
8	Patapsco	130
9	Cold Spring Lane	140
10	West Cold Spring	140
11	Lutherville	150
12	Cherry Hill	150
13	Mt Washington	160
14	Cromwell / Glen Burnie	160
15	Rogers Avenue	160
16	Falls Road	180
17	Hamburg Street	180
18	Milford Mill	180
19	Upton	180
20	Hunt Valley	200
21	University of Baltimore	200
22	Centre Street	200
23	Old Court	200
24	Penn North	200
25	Reisterstown Plaza	220
26	State Center	220
27	Shot Tower	220
28	North Avenue	250
29	Mondawmin	250
30	Johns Hopkins Hospital	280
31	Convention Center	300
32	BWI Airport	300
33	Owings Mills	300
34	Charles Center	300
35	Lexington Market	350
36	Penn Station	400
37	Camden Yards	500

5. Screenshots (continued)

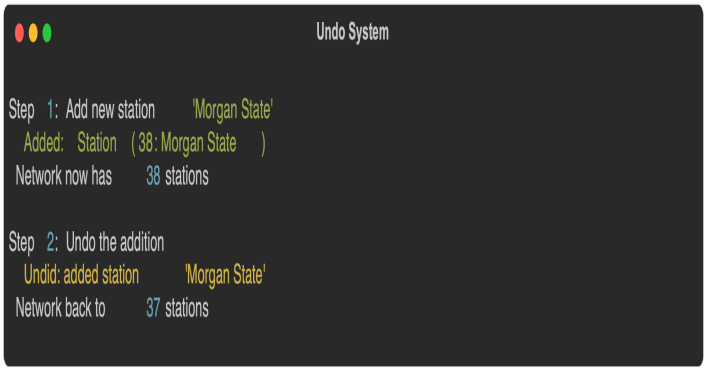
Binary vs Linear Search



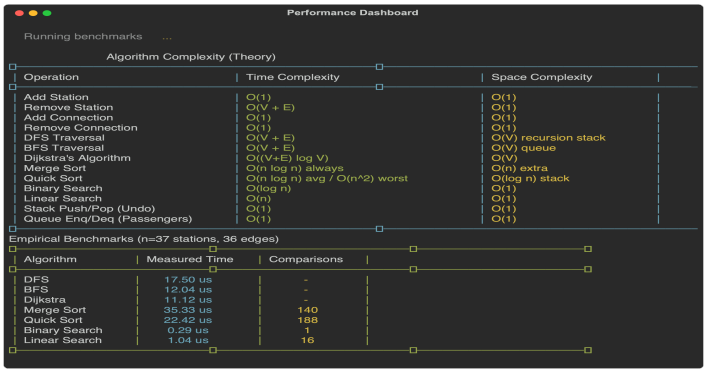
Passenger Boarding (Queue)



Undo System (Stack)



Performance Dashboard



6. Conclusion

This project demonstrates the practical application of core data structures and algorithms to a real-world transit network. By using actual Baltimore MTA station data (37 stations across Light Rail and Metro SubwayLink) rather than abstract examples, the project grounds algorithmic concepts in a tangible system.

Key takeaways:

1. Graph: An adjacency list (dict of dicts) provides O(1) edge operations and naturally models the sparse connectivity of a transit network.
2. DFS vs BFS: BFS guarantees the shortest unweighted path and typically visits fewer nodes, while DFS explores deeply first. Both run in O(V+E) time.
3. Dijkstra: The min-heap priority queue enables weighted shortest path in O((V+E) log V). Cross-line routing through Lexington Market demonstrates real transfers.
4. Sorting: Merge Sort's guaranteed O(n log n) and stability costs O(n) space. Quick Sort's in-place operation gives better cache performance despite O(n^2) worst case.
5. Searching: Binary Search's O(log n) dramatically outperforms Linear Search's O(n), confirmed with 3 vs 20 comparisons on 37 stations.
6. Queue/Stack: The deque-based passenger queue and list-based undo stack demonstrate fundamental FIFO/LIFO patterns with O(1) operations.

The Rich terminal UI elevates the project with animated algorithm visualizations and a performance dashboard that bridges Big-O theory with measured benchmarks, reinforcing that algorithmic analysis predicts real-world behavior.

The simulator uses real Baltimore MTA data: 24 Light Rail stations from Hunt Valley to Cromwell/Glen Burnie (with BWI Airport spur), 14 Metro stations from Owings Mills to Johns Hopkins Hospital, connected at Lexington Market. All 37 stations with 36 weighted edges and travel times of 2-5 minutes.

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

[2] Baltimore MTA. Maryland Transit Administration System Map. mta.maryland.gov

[3] Python Software Foundation. collections.deque, heapq documentation. docs.python.org

[4] Will McGugan. Rich: Python library for rich text in terminal. github.com/Textualize/rich